



Natural Language Processing: Transformers

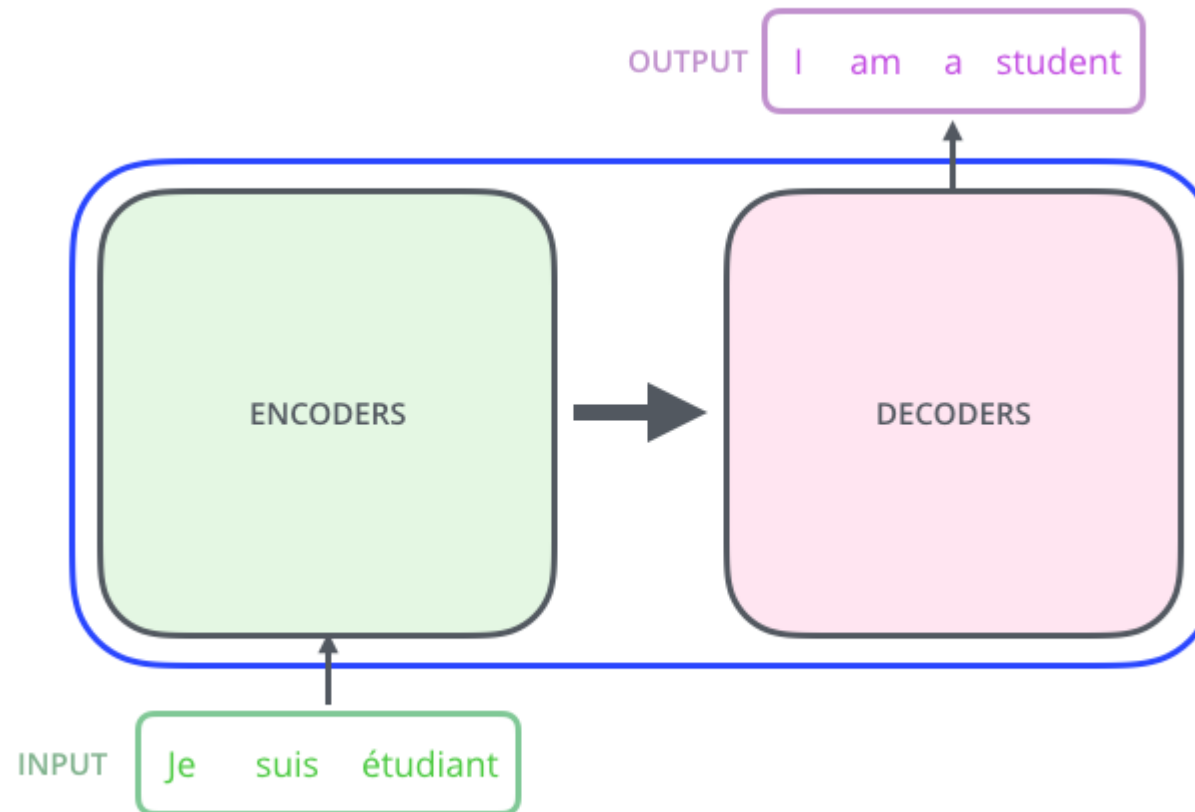
HSE Faculty of Computer Science

Murat Khazhgeriev

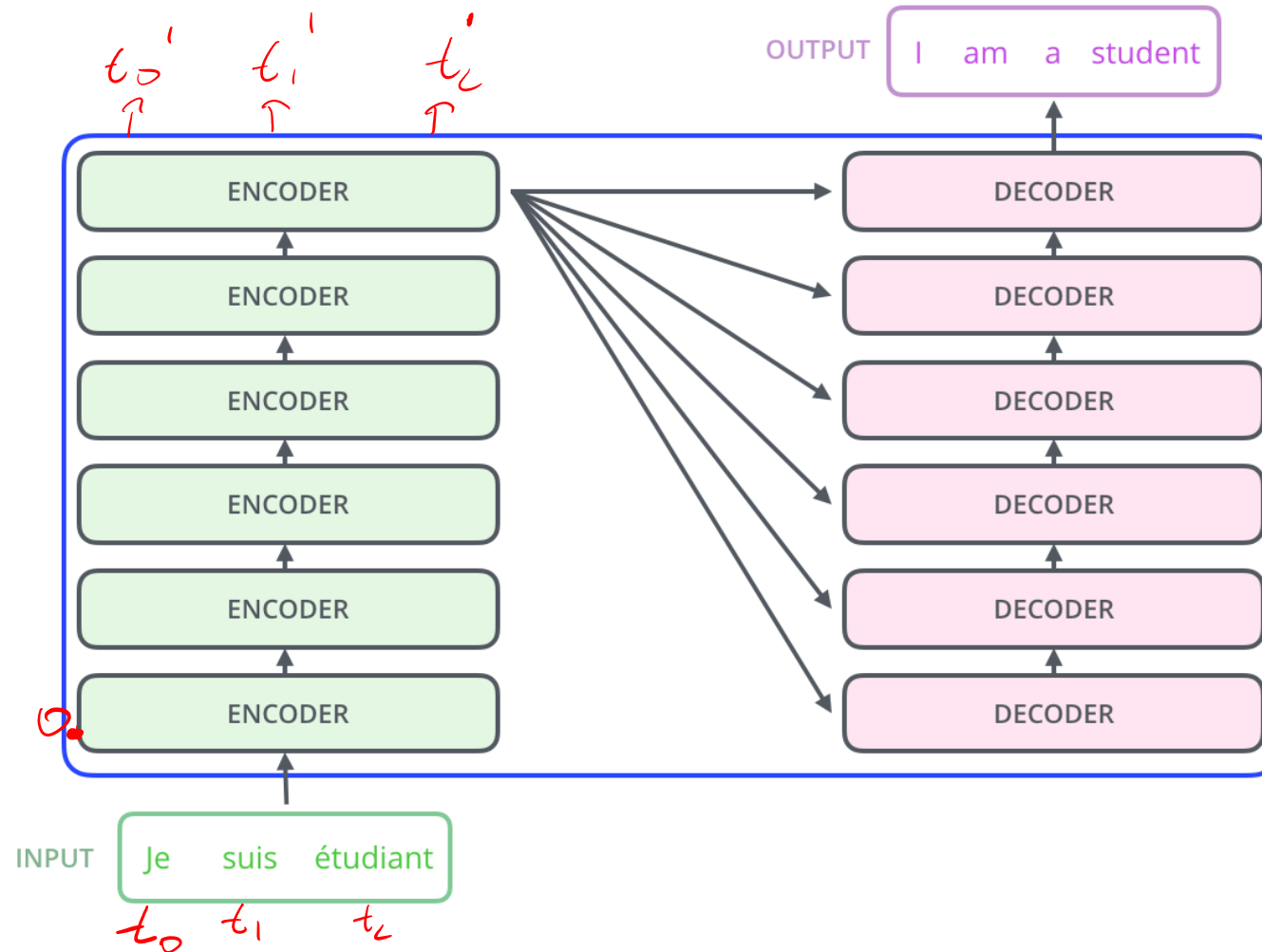
Transformer is an example of Encoder-Decoder architecture



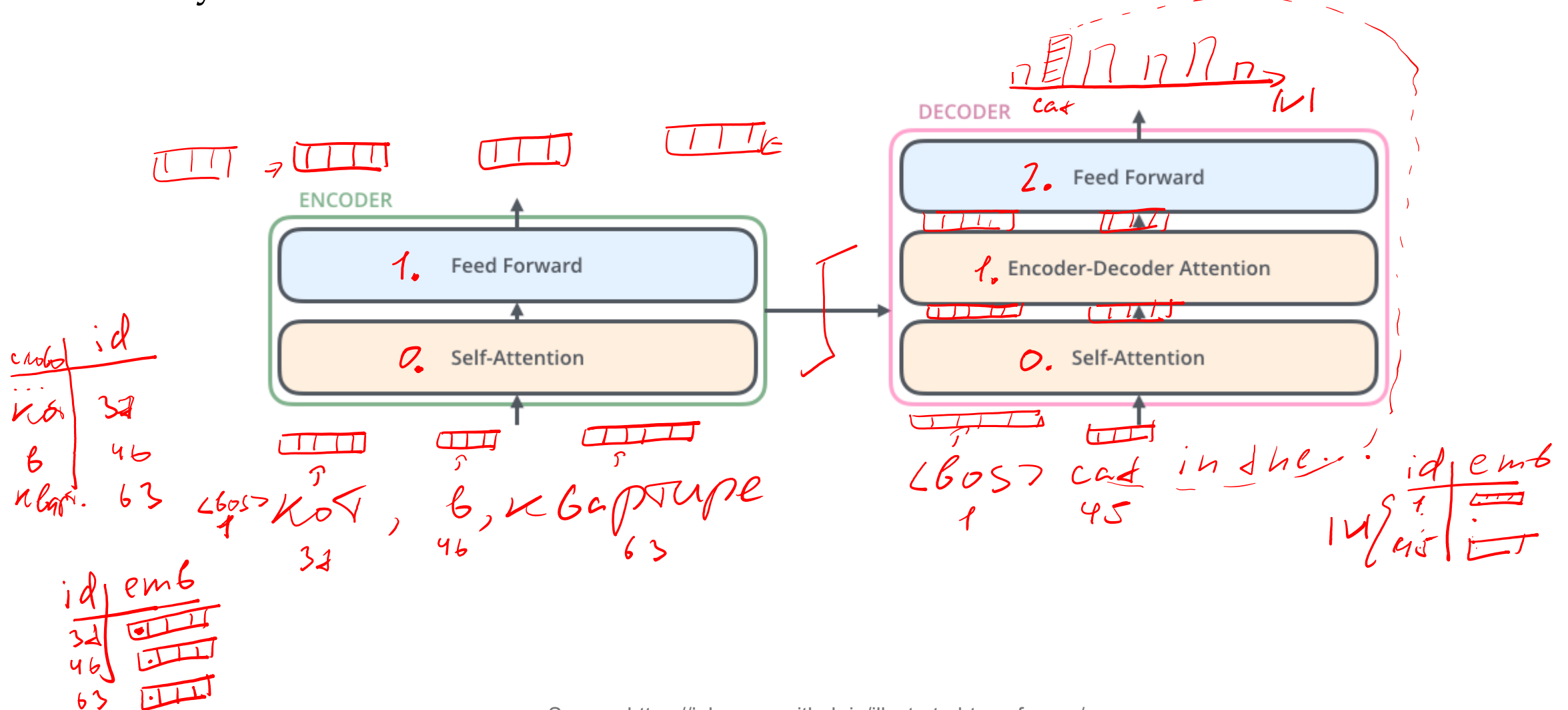
Transformer is an example of Encoder-Decoder architecture



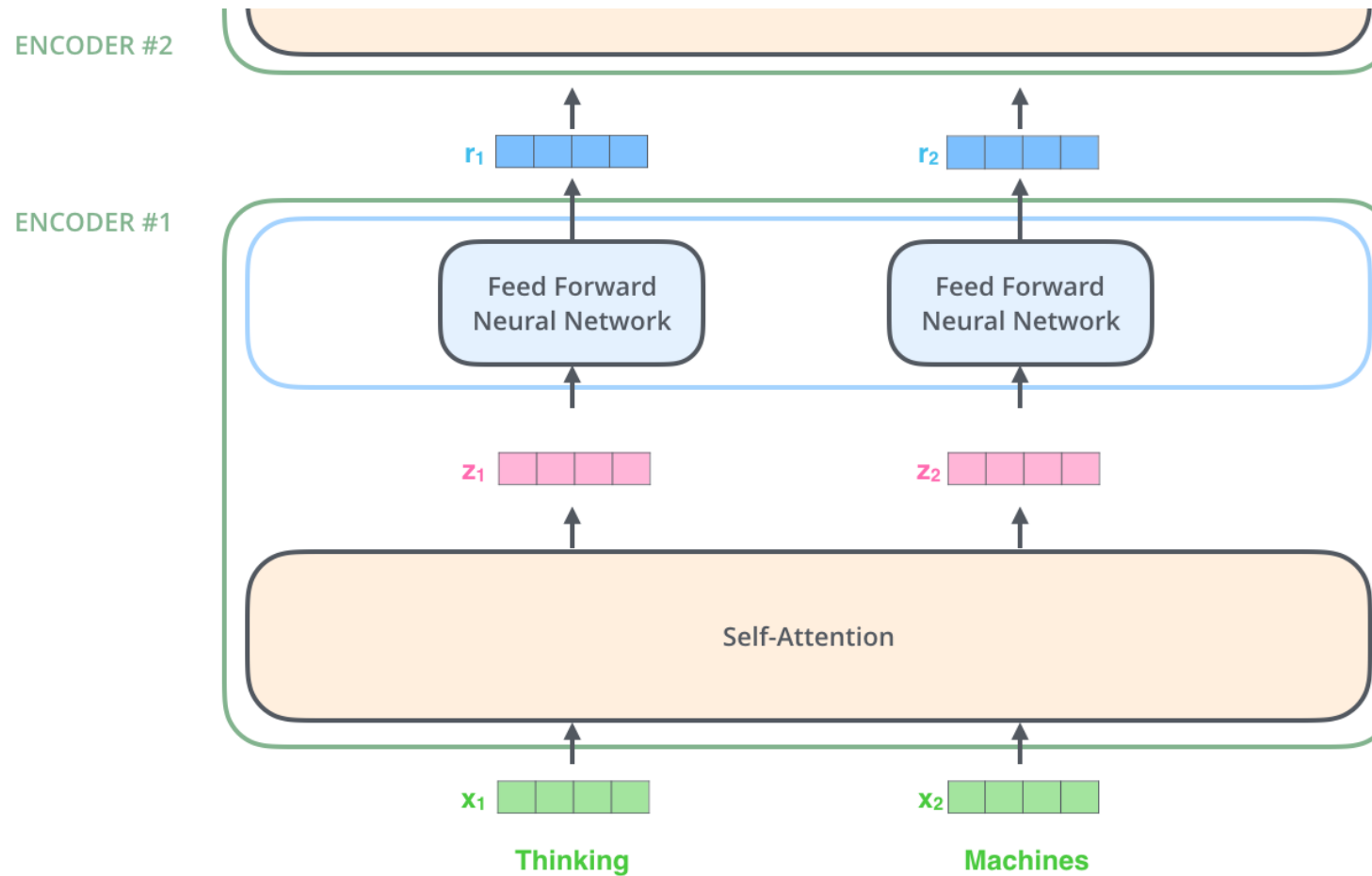
Transformer is an example of Encoder-Decoder architecture



Always two there are. No more, no less. An Attention and an FFN. © Yoda



Always two there are. No more, no less. An Attention and an FFN. © Yoda



Inside a Self-Attention

$$\begin{matrix} x_1 & \boxed{} & \boxed{} & \boxed{} & \boxed{} \\ x_2 & \boxed{} & \boxed{} & \boxed{} & \boxed{} \end{matrix} \rightarrow \begin{bmatrix} -x_1- \\ -x_2- \end{bmatrix}$$

w^q → query
 w^k → key
 w^v → values



Input

Embedding

Queries

Keys

Values

Score

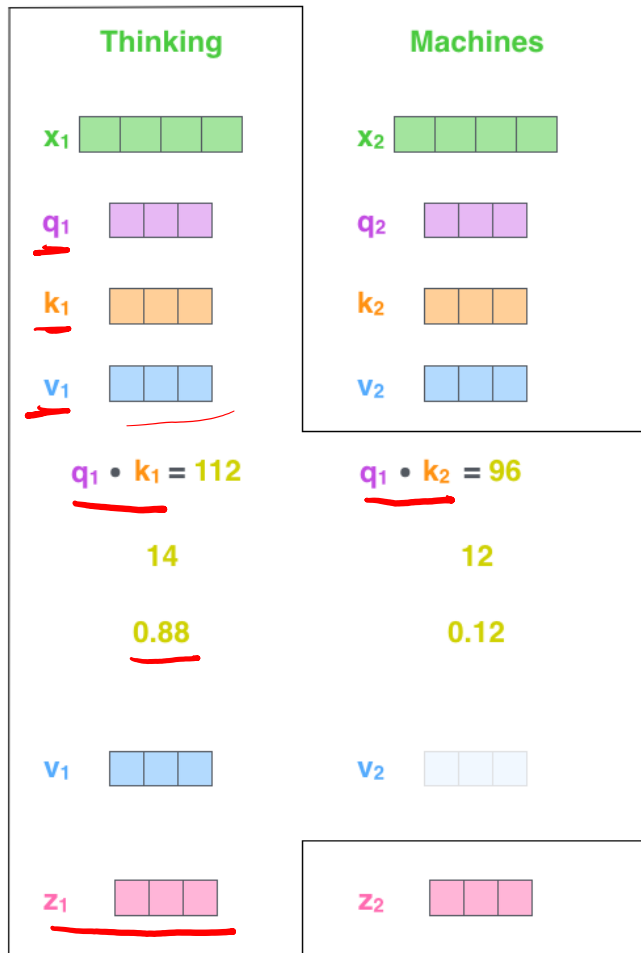
Divide by 8 ($\sqrt{d_k}$)

Softmax

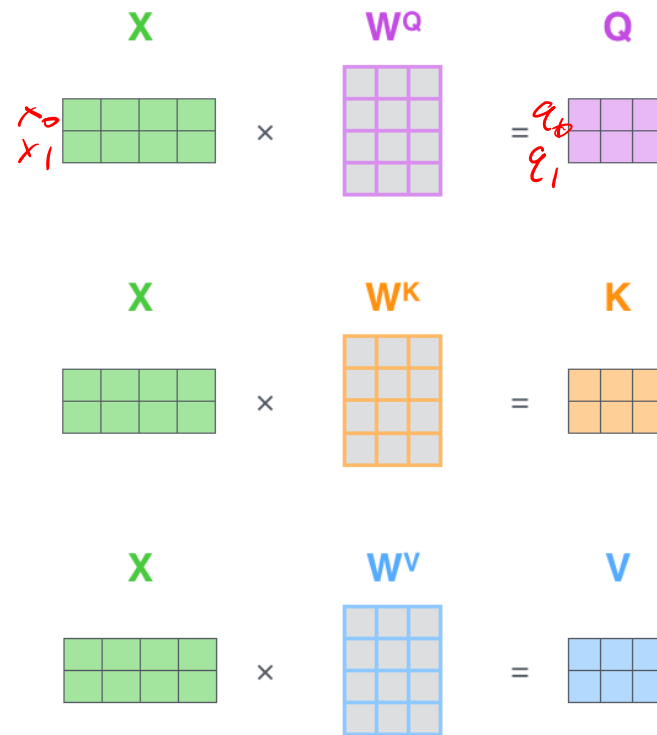
Softmax

X
Value

Sum



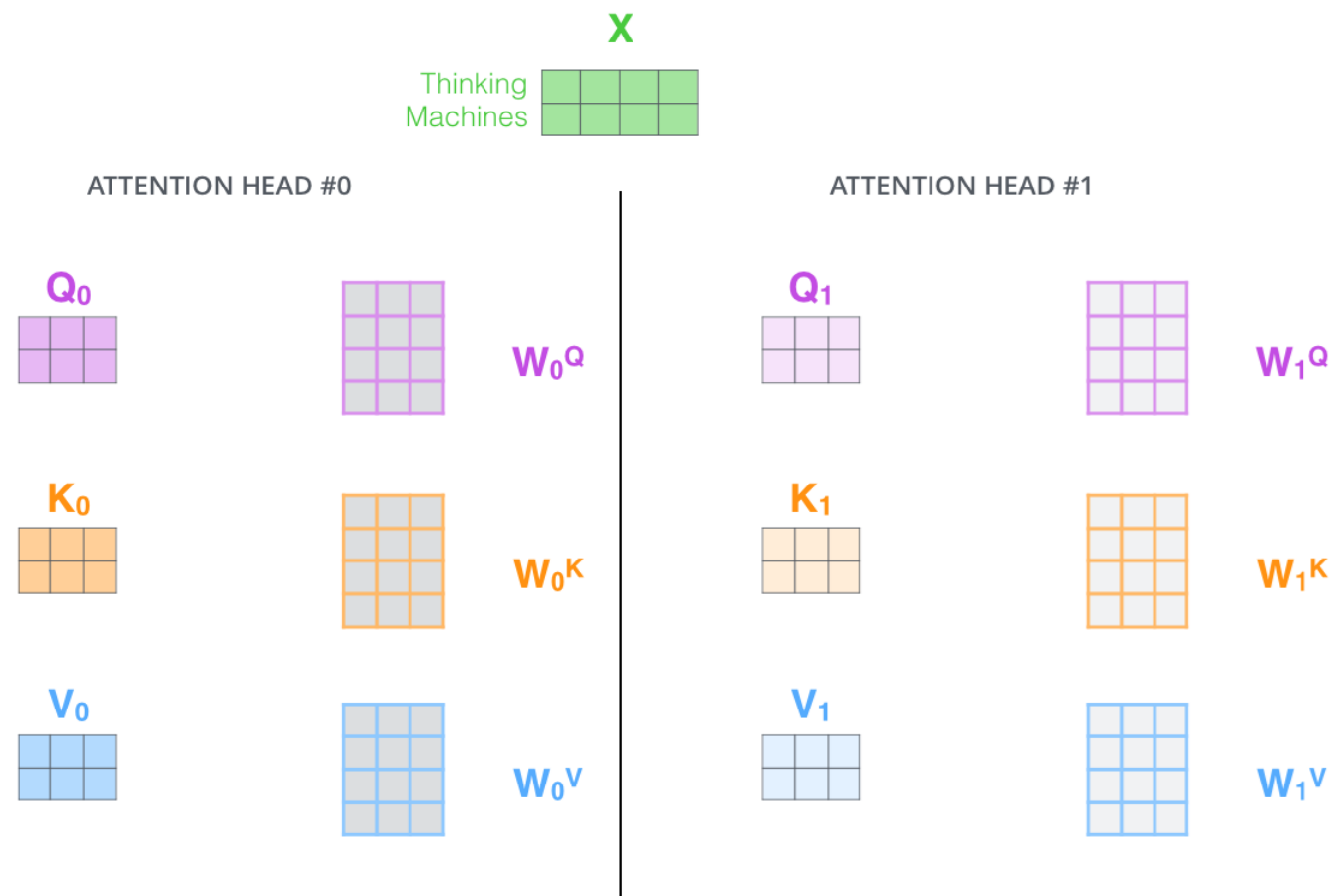
Inside a Self-Attention: Matrix View



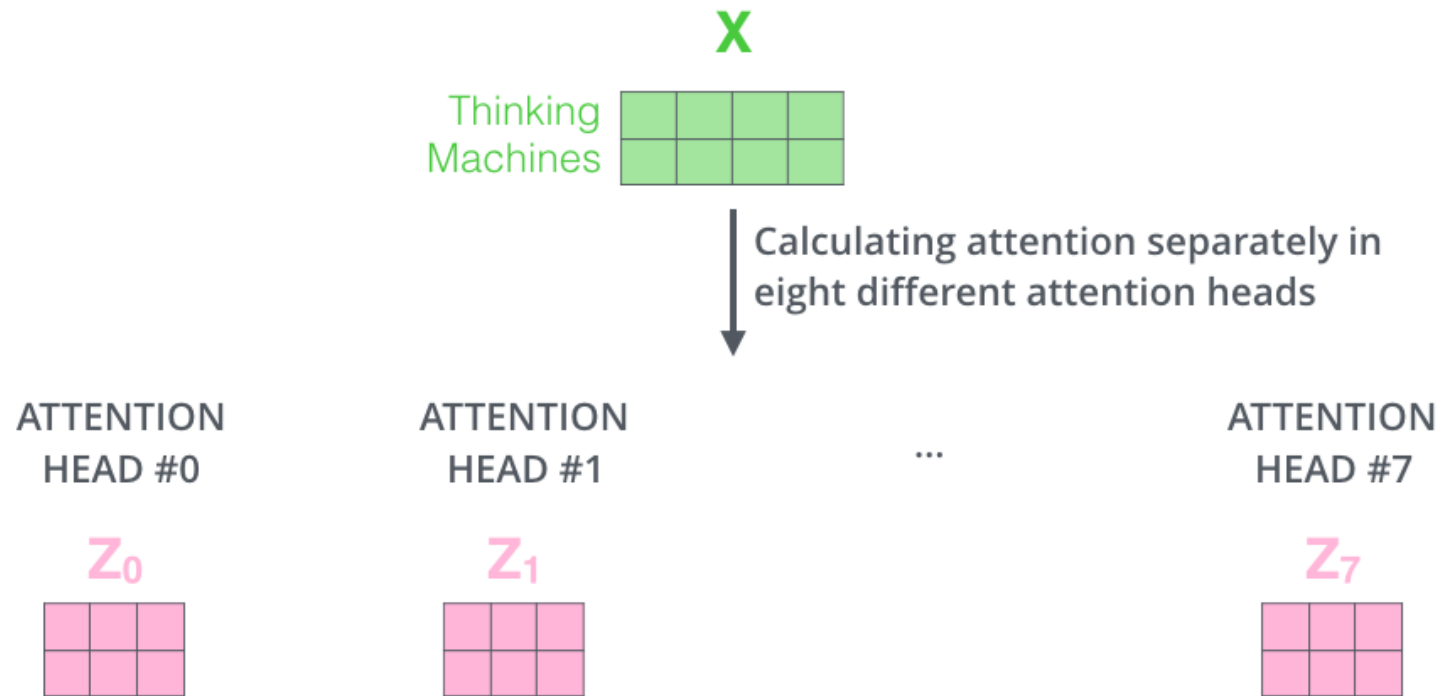
Inside a Self-Attention: Matrix View

$$\text{softmax} \left(\frac{\overset{\text{Q}}{\begin{matrix} q_0 \\ q_1 \end{matrix} \begin{array}{|c|c|c|} \hline \square & \square & \square \\ \hline \square & \square & \square \end{array}} \times \overset{\text{K}^T}{\begin{matrix} k_0 \ k_1 \end{matrix} \begin{array}{|c|c|} \hline \square & \square \\ \hline \square & \square \\ \hline \square & \square \end{array}} \right) \overset{\text{V}}{\begin{matrix} v_0 \\ v_1 \end{matrix} \begin{array}{|c|c|c|} \hline \square & \square & \square \\ \hline \square & \square & \square \end{array}}$$
$$= \overset{\text{Z}}{\begin{array}{|c|c|c|} \hline \square & \square & \square \\ \hline \square & \square & \square \end{array}}$$

A beast with many heads

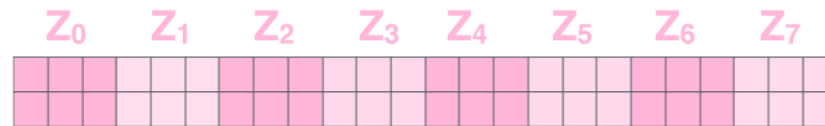


A beast with many heads



A beast with many heads

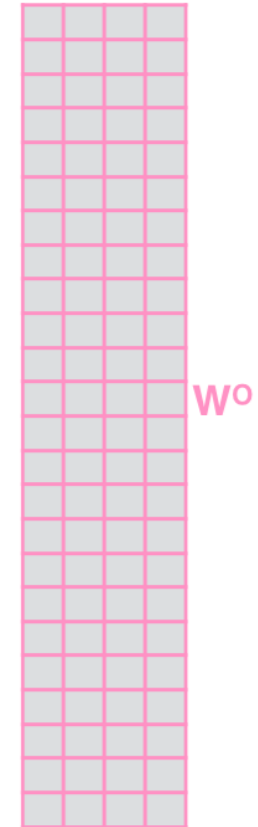
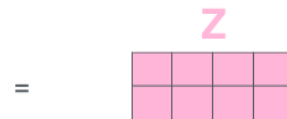
1) Concatenate all the attention heads



2) Multiply with a weight matrix W^O that was trained jointly with the model

X

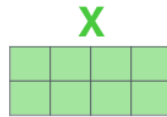
3) The result would be the Z matrix that captures information from all the attention heads. We can send this forward to the FFNN



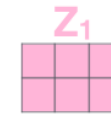
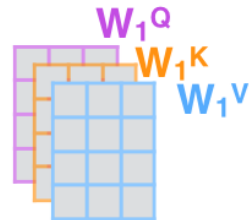
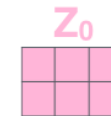
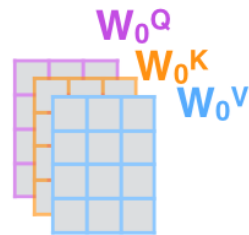
A multi-head attention overview

- 1) This is our input sentence*
- 2) We embed each word*
- 3) Split into 8 heads. We multiply X or R with weight matrices
- 4) Calculate attention using the resulting $Q/K/V$ matrices
- 5) Concatenate the resulting Z matrices, then multiply with weight matrix W^O to produce the output of the layer

Thinking
Machines



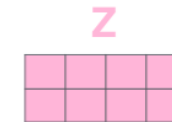
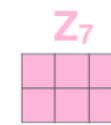
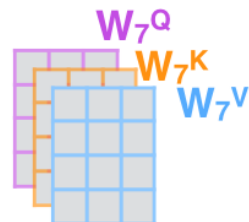
* In all encoders other than #0, we don't need embedding. We start directly with the output of the encoder right below this one



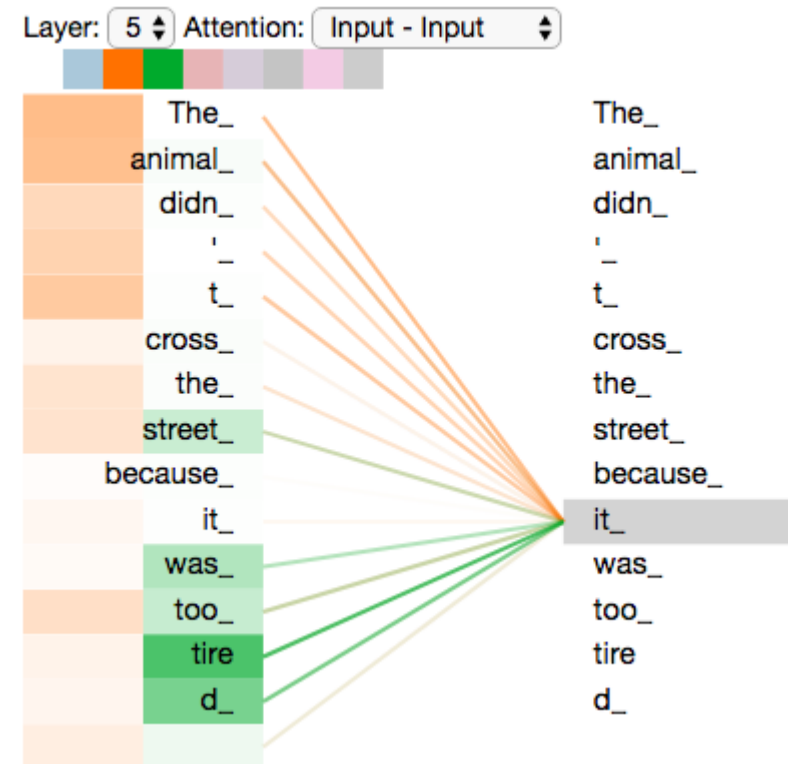
...

...

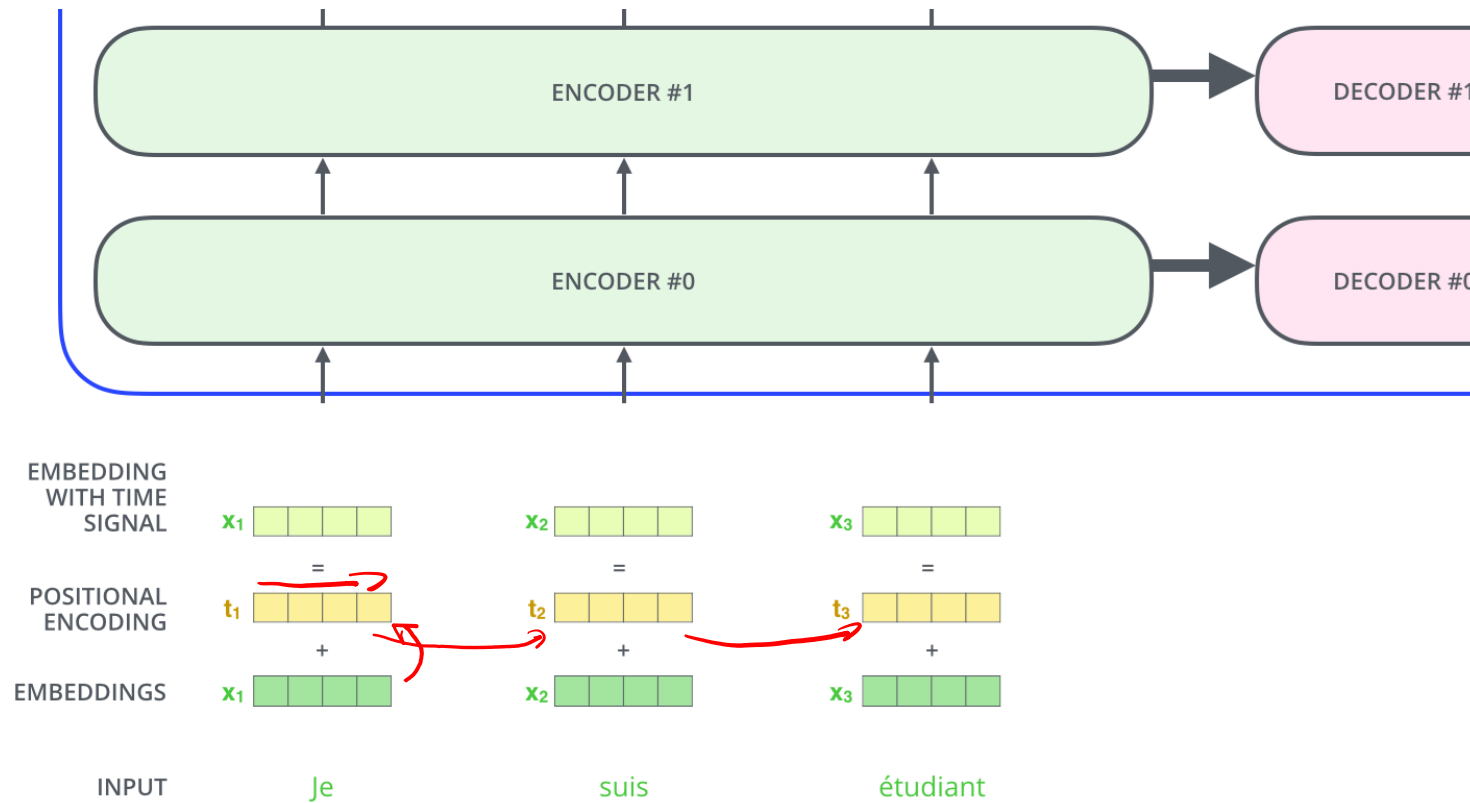
...



Each head focuses on a specific representation



As opposed to RNNs, Transformers do not track the position implicitly

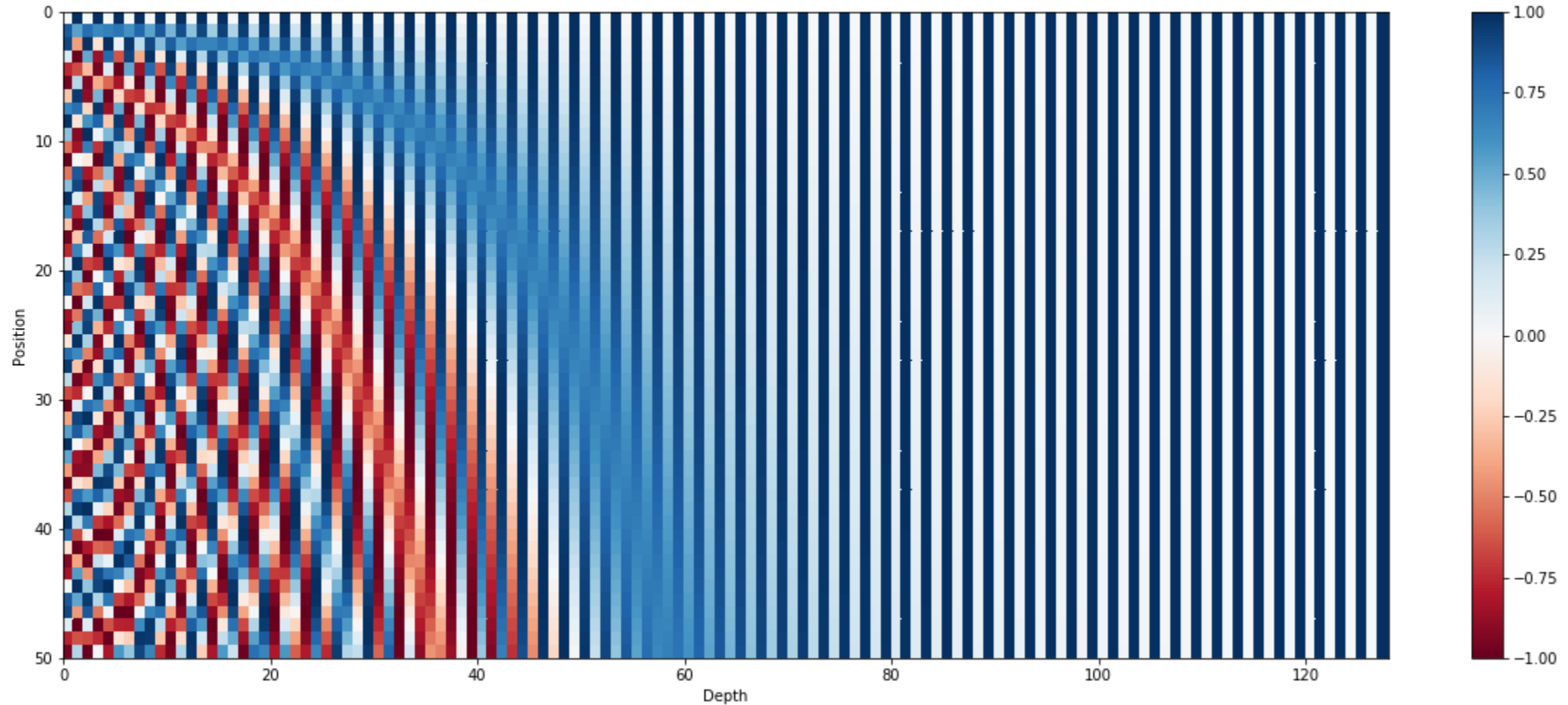


Absolute Positional Encoding

$$\text{PE}(\text{pos}, 2i) = \sin\left(\text{pos}/10000^{2i/d_{\text{model}}}\right)$$

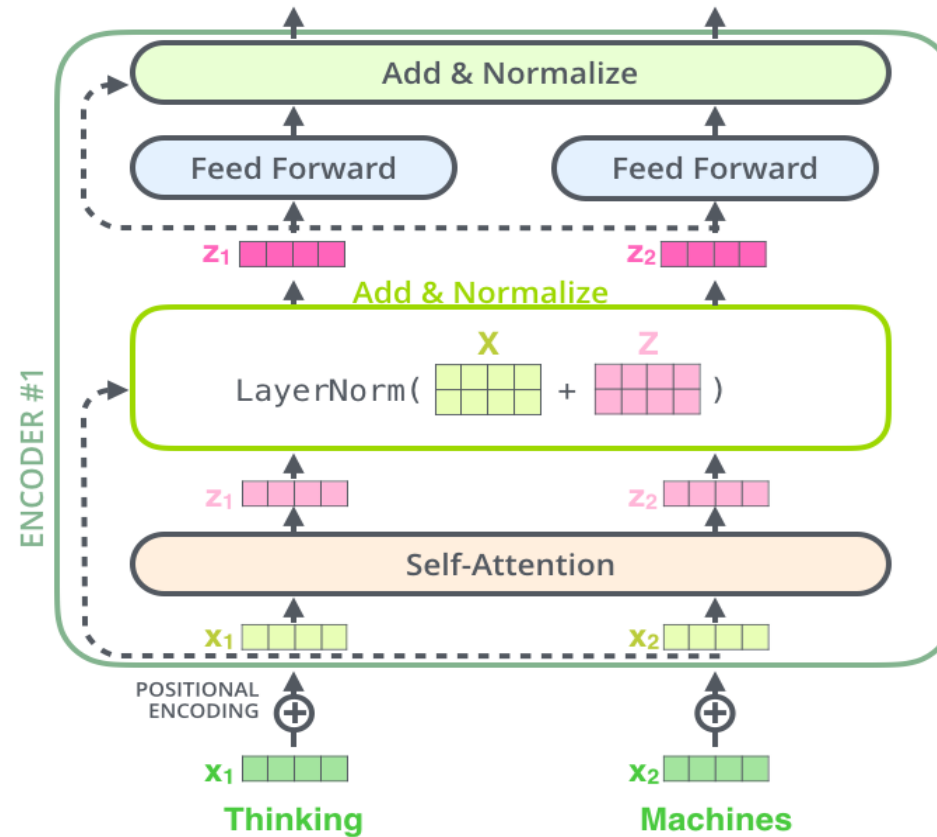
$$\text{PE}(\text{pos}, 2i + 1) = \cos\left(\text{pos}/10000^{2i/d_{\text{model}}}\right)$$

What it looks like

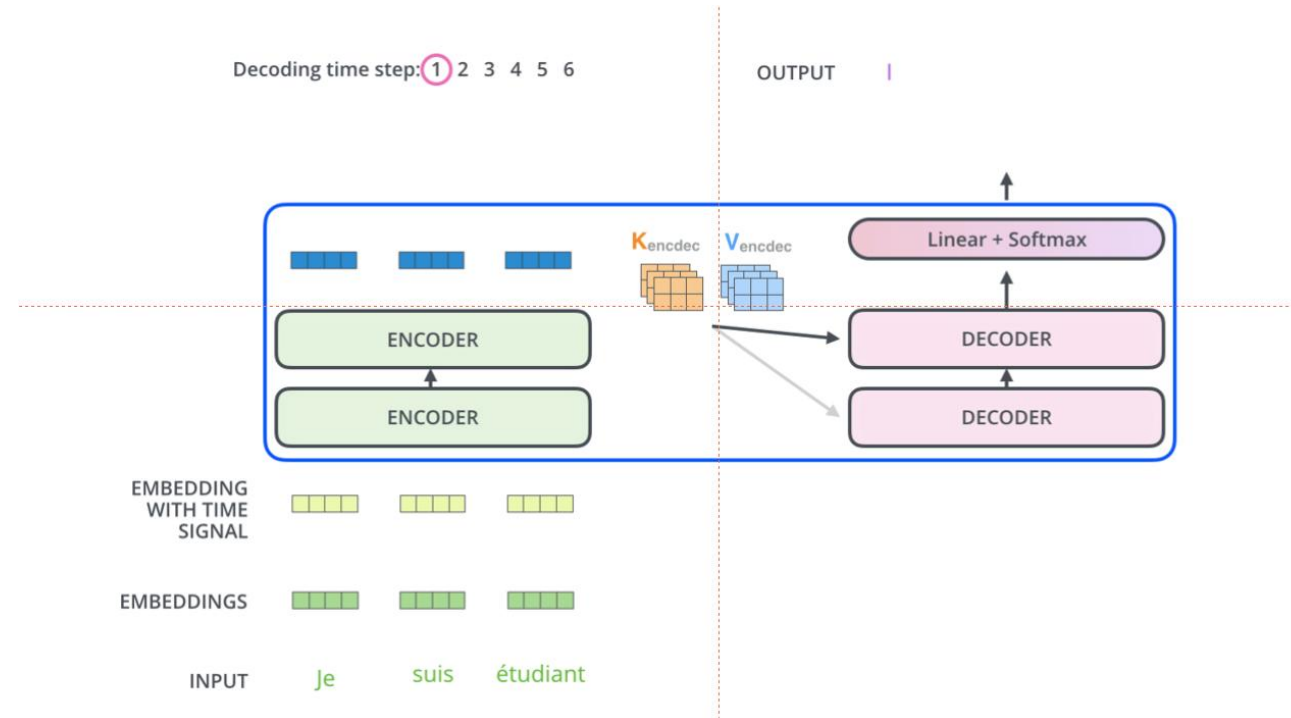


Source: https://kazemnejad.com/blog/transformer_architecture_positional_encoding/

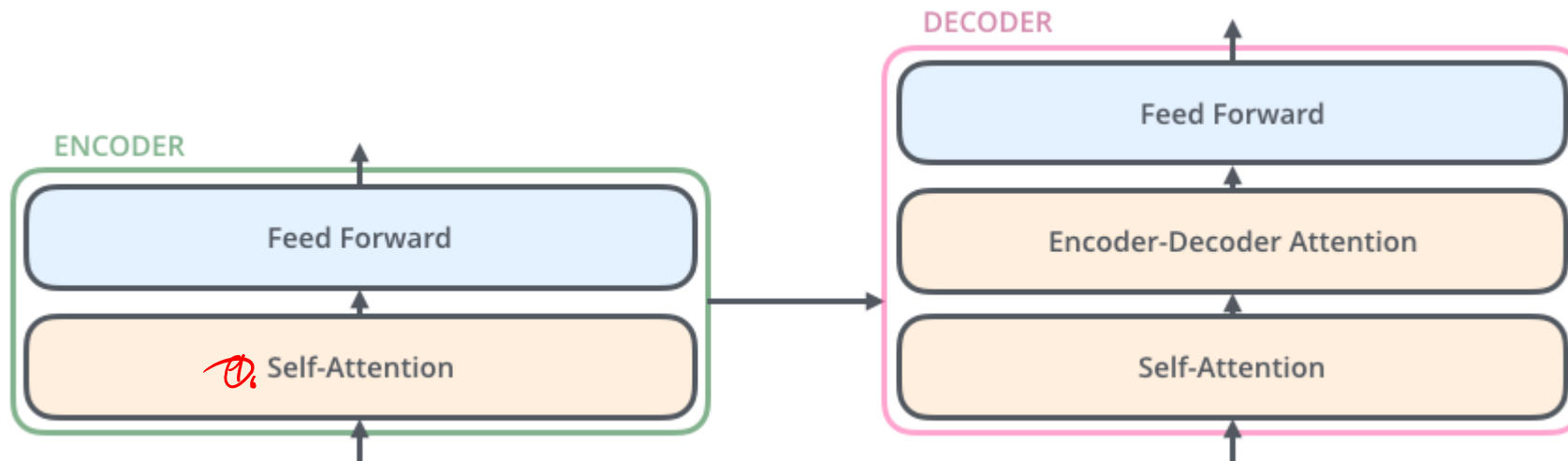
Skip Connection and Layer Normalization for a robust training



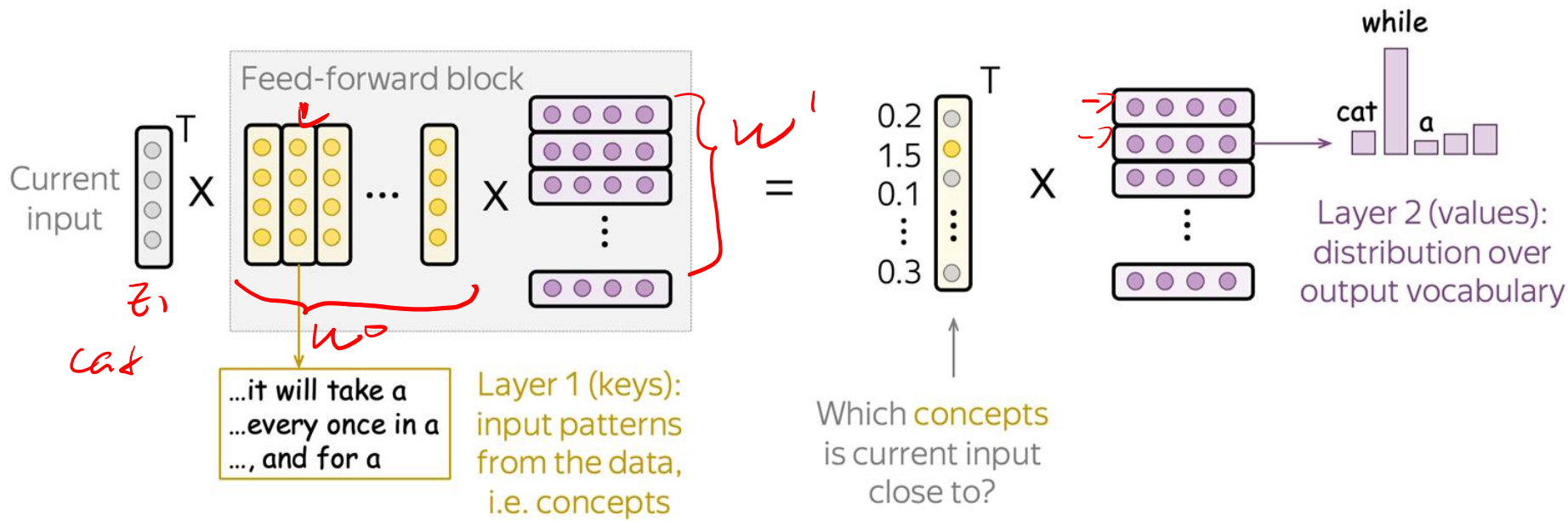
Bringing it all together



Always two there are. No more, no less. An Attention and an FFN. © Yoda



Attention is all you need (but not really)

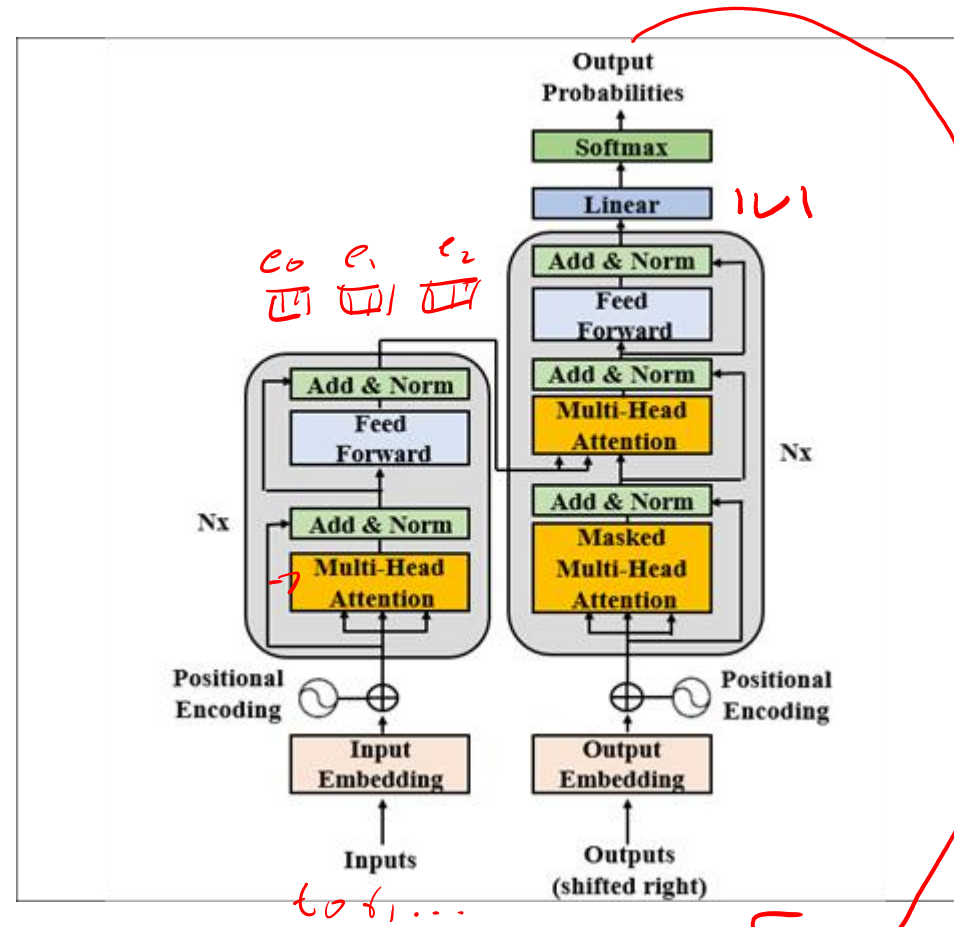


$$z = \begin{bmatrix} -z_0 \\ -z_1 \\ \vdots \end{bmatrix}$$

$$\Delta(z W^0) W^1$$

$$\begin{bmatrix} c_0 & c_1 & \dots \end{bmatrix} \begin{bmatrix} -z_0 \\ -z_1 \\ \vdots \end{bmatrix}$$

Attention is all you need



Handwritten notes illustrating the relationship between input and output embeddings and weights:

$x_0 \rightarrow W^e \rightarrow e$

$e_0, e_1, e_2 \rightarrow W^k \rightarrow k$

$x_0, x_1, x_2 \rightarrow W^v \rightarrow v$

